ARTIFICIAL INTELLIGENCE CSA1701 LAB OBSERVATON
Mohammed Fareeth S A (192511288)

EXPERIMENT 10:

Aim

To implement and solve the classical **8-Puzzle Problem** using the *A Search** algorithm with the Manhattan distance heuristic in order to find the optimal sequence of moves from an initial board configuration to the goal state.

Algorithm

1. **Define State Representation:** Represent the 3x3 puzzle board as a tuple of tuples (or a flattened tuple) containing numbers 0 through 8 (where 0 represents the blank space).
2. **Define Heuristic Function:** Use **Manhattan Distance**, which calculates the sum of the absolute distances from the current position of each tile to its goal position.
3. **Initialize Priority Queue:** Create a priority queue (min-heap) to store search nodes. Each entry stores:
    - Total estimated cost $f(n) = g(n) + h(n)$
    - Cost from start $g(n)$
    - Current board state
    - Path of moves taken to reach the state
4. *Search Loop (A):**
    - Pop the state with the lowest $f(n)$ value from the priority queue.
    - If the current state matches the goal state, return the path of moves.
    - Otherwise, generate valid successor states by sliding the blank space (0) Up, Down, Left, or Right.
5. **Evaluate Successors:** Calculate $g(n)$ (cost + 1) and $h(n)$ (Manhattan distance) for each successor, then push them onto the priority queue.
6. **Terminate:** Repeat until the goal configuration is reached and output the optimal moves and total step cost.

Code:

```
import heapq
class PuzzleAStar:
    def __init__(self, initial_state, goal_state):
        self.initial_state = initial_state
        self.goal_state = goal_state
        self.goal_pos = {val: (r, c) for r, row in enumerate(goal_state) for c, val in enumerate(row)}
    def manhattan_distance(self, state):
        distance = 0
        for r in range(3):
            for c in range(3):
                val = state[r][c]
                if val != 0:
                    goal_r, goal_c = self.goal_pos[val]
                    distance += abs(r - goal_r) + abs(c - goal_c)
        return distance
    def get_successors(self, state):
        successors = []
        for r in range(3):
            for c in range(3):
                if state[r][c] == 0:
                    blank_r, blank_c = r, c
                    break
        moves = [('Up', -1, 0), ('Down', 1, 0), ('Left', 0, -1), ('Right', 0, 1)]

        for move, dr, dc in moves:
            nr, nc = blank_r + dr, blank_c + dc
            if 0 <= nr < 3 and 0 <= nc < 3:
                new_state = [list(row) for row in state]
                new_state[blank_r][blank_c], new_state[nr][nc] = new_state[nr][nc], new_state[blank_r][blank_c]
                successors.append((move, tuple(tuple(row) for row in new_state)))
        return successors
    def search(self):
        initial_h = self.manhattan_distance(self.initial_state)
        pq = [(initial_h, 0, self.initial_state, [])]
        explored = set()
        while pq:
            f, g, current_state, path = heapq.heappop(pq)
            if current_state == self.goal_state:
                return path, g
            if current_state in explored:
                continue
            explored.add(current_state)
            for move, next_state in self.get_successors(current_state):
                if next_state not in explored:
                    g_new = g + 1
                    h_new = self.manhattan_distance(next_state)
                    f_new = g_new + h_new
                    heapq.heappush(pq, (f_new, g_new, next_state, path + [move]))
        return None, -1
initial_board = (
    (1, 2, 3),
    (4, 0, 5),
```

```
============= RESTART: C
Initial Board State:
(1, 2, 3)
(4, 0, 5)
(7, 8, 6)

Goal State Reached!

Optimal Action Sequence:
1. Slide Right
2. Slide Down

Total Cost (Steps): 2
```

Result

The A* search algorithm successfully evaluated the state space using the Manhattan distance heuristic function. Starting from the initial 8-puzzle board configuration, the algorithm efficiently pruned sub-optimal paths and computed the shortest sequence of slide movements required to reach the target goal state.

Conclusion

The program successfully demonstrated the power of heuristic search algorithms in solving complex combinatorial problems like the 8-Puzzle. By combining the actual path cost $g(n)$ with an admissible heuristic $h(n)$ (Manhattan distance), the A* algorithm guaranteed an optimal solution with significantly fewer explored states compared to uninformed search strategies like BFS or DFS